

Name of the Student	G.Yaswanth Reddy
Reg. No	192425267
GitHub Link	https://github.com/yaswanth-210

SIMATS ENGINEERING

CO2 - ASSESSMENT TOOL 2

Scenario-Based Questions

COURSE NAME: Deep Learning

COURSE CODE: MLA04

COURSE OUTCOME COVERED

CO 2:Students will be able to apply supervised learning algorithms such as linear regression, classification models, decision trees, neural networks, support vector machines, and ensemble methods to solve real-world prediction and classification problems. They will gain the ability to select appropriate models, train them using datasets, and evaluate their performance. Students will also understand the strengths and limitations of different supervised learning approaches.(BTL-4)

CO Weightage : 15%

Assessment Tool Weightage: 40%

Duration: 90 minutes

Marks: 25

Code of Conduct:

I G.Yaswanth Reddy(1924252267) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

G.Yaswanth Reddy
Signature of the student:

G.Yaswanth Reddy
Name of the student:

Scenario-Based Questions

CO-AT2

QUESTIONS: 05

Total Marks:25

1. Unsupervised Training of Neural Networks, Auto Encoders

A hospital wants to identify abnormal ECG patterns from thousands of unlabeled records. Which deep learning technique from the syllabus would you recommend? Justify your choice and explain the workflow.

2. Auto Encoders, Representation Learning

A retail company wants to reduce the dimensionality of customer purchase data before performing customer segmentation. Explain how an autoencoder can be used to solve this problem.

3. Width and Depth of Neural Networks, Activation Functions

A facial recognition system performs poorly when trained using a shallow neural network. Recommend architectural changes involving network depth and activation functions.

4. Restricted Boltzmann Machines (RBMs), Unsupervised Training of Neural Networks

A streaming service wants to recommend movies to users based on their viewing history without using labeled data. Explain how RBMs can be applied to build the recommendation engine.

5. Deep Learning Applications, Representation Learning

A manufacturing company wants to detect defective products from sensor readings collected every second. Design a deep learning solution using concepts from representation learning and neural networks.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and accurate	Shows correct relationships	Relationships are weak, unclear, or	No meaningful relationships established

		relationships between concepts	with minor gaps	partially incorrect	
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand

SCENARIO-BASED ASSESSMENT

Deep Learning and Neural Networks

1. Unsupervised Training of Neural Networks – Autoencoders

Scenario

A hospital has collected thousands of ECG (Electrocardiogram) records from patients. However, most of the records are unlabeled, meaning doctors have not manually marked every record as normal or abnormal. The hospital wants to automatically identify unusual ECG patterns.

Recommended Technique

An Autoencoder is a suitable deep learning technique for this problem because it can learn the important patterns in unlabeled data without requiring abnormality labels. It learns to compress an input ECG signal into a lower-dimensional representation and then reconstruct the original signal.

Justification

- It does not require a large set of manually labeled ECG records.
- It can learn the normal structure and patterns present in ECG signals.
- Unusual ECG patterns can be detected using reconstruction error.
- It can automatically learn useful features from raw ECG data.
- It is suitable for representation learning and anomaly detection.

Workflow

1. ECG Data Collection – Collect thousands of ECG records from the hospital.
2. Preprocessing – Remove noise, handle missing values, normalize the signals and divide them into suitable time windows.
3. Encoder – Feed the processed ECG signal into an encoder network that converts the input into a compact latent representation.
4. Latent Representation – The compressed representation captures important characteristics of the ECG signal.
5. Decoder – The decoder reconstructs the ECG signal from the latent representation.
6. Reconstruction Error – Compare the original ECG signal with the reconstructed signal.

7. Threshold Selection – Establish a suitable reconstruction-error threshold using validation data.
8. Abnormality Detection – ECG records with unusually high reconstruction error are flagged for further medical examination.

Workflow Representation

ECG Records → Preprocessing → Encoder → Latent Representation → Decoder → Reconstruction → Error Calculation → Abnormal Pattern Detection

Conclusion

Therefore, an Autoencoder is recommended because it can perform unsupervised representation learning and identify abnormal ECG patterns without requiring every training record to be labeled.

2. Autoencoders – Representation Learning

Scenario

A retail company has customer purchase data containing hundreds of variables such as products purchased, purchase frequency, spending amount, category preferences and transaction behavior. The company wants to reduce the dimensionality of this data before performing customer segmentation.

Solution Using an Autoencoder

An Autoencoder can learn a compact representation of the high-dimensional customer purchase data. Instead of manually selecting features, the network learns the most useful patterns automatically.

Architecture

Input Data → Encoder → Bottleneck/Latent Representation → Decoder → Reconstructed Data

Working

9. Prepare the customer purchase dataset and normalize numerical features.
10. Feed the high-dimensional purchase vector into the encoder.
11. The encoder gradually reduces the number of features and produces a compact latent vector.
12. The latent vector contains the important characteristics of customer purchasing behavior.
13. The decoder attempts to reconstruct the original purchase data from the latent vector.

14. Train the network by minimizing the reconstruction error between the original and reconstructed data.
15. After training, discard or bypass the decoder and use the latent representations as reduced-dimensional customer features.
16. Apply a clustering algorithm such as K-Means to the latent representations to create customer segments.

Example

If the original customer dataset contains 100 features, the Autoencoder might learn a latent representation containing only 10 or 20 features. These learned features may represent hidden patterns such as spending behavior, product preferences and purchase frequency.

Benefits

- Reduces the number of dimensions.
- Removes redundant information.
- Learns features automatically.
- Makes clustering easier and potentially more efficient.
- Can improve visualization when the latent representation has two or three dimensions.

Conclusion

The Autoencoder acts as a representation-learning mechanism that converts complex purchase data into a compact form. The resulting latent features can then be used as input to a customer-segmentation algorithm.

3. Width and Depth of Neural Networks – Activation Functions

Scenario

A facial recognition system performs poorly because it uses a shallow neural network with only one or two hidden layers. The system struggles to distinguish faces under different lighting, poses and expressions.

Recommended Architectural Changes

The system should be redesigned using a deeper neural network with an appropriate number of neurons in each layer and suitable nonlinear activation functions. For image-based facial recognition, a Convolutional Neural Network (CNN) would be a strong architecture.

Increasing Network Depth

Depth refers to the number of layers in a neural network. A deeper network can learn hierarchical representations. For facial recognition, early layers can learn simple patterns such as edges, while deeper layers can learn eyes, noses, facial structures and complete facial features.

Example hierarchy:

- Early layers → edges and basic textures
- Middle layers → eyes, nose, mouth and other facial components
- Deeper layers → complex facial structures and identity-related patterns

Adjusting Network Width

Width refers to the number of neurons or feature maps in a layer. Increasing width can give the network greater capacity to learn different features. However, excessive width increases the number of parameters, memory usage and risk of overfitting.

Activation Function Recommendation

ReLU is a suitable default activation function for hidden layers because it is computationally efficient and introduces non-linearity. If neurons frequently become inactive because of negative inputs, Leaky ReLU can be used because it allows a small gradient for negative values.

Activation Options

Activation	Behavior	Advantage	Use
ReLU	$\max(0, x)$	Simple and fast	Default hidden-layer activation
Leaky ReLU	x for positive; αx for negative	Reduces dying neurons	When ReLU neurons become inactive
ELU	x for positive; $\alpha(e^x - 1)$ for negative	Smooth negative behavior	When smoother negative activation is useful

Recommended Architecture

Input Face Image → Convolution/Feature Extraction Layers → Deeper Hidden Layers with ReLU/LReLU → Fully Connected Layer → Face Embedding → Classification or Similarity Matching

Conclusion

The poor performance of the shallow network can be addressed by increasing depth, selecting suitable width, using nonlinear activation functions and applying a CNN-based architecture. The network should be deep enough to learn complex facial representations without becoming unnecessarily large.

4. Restricted Boltzmann Machines (RBMs) – Unsupervised Training

Scenario

A streaming service wants to recommend movies to users based on their viewing history. It has information about which users watched, liked or interacted with movies, but it does not have a manually labeled training set specifying the correct recommendation for every user.

Using an RBM

A Restricted Boltzmann Machine can learn hidden relationships between users and movies from their interaction data. RBMs are generative neural networks containing a visible layer and a hidden layer.

RBM Structure

Visible Layer: Movie interaction data (watched/liked/not watched)

Hidden Layer: Latent user-preference patterns

There are connections between visible and hidden units, but no connections between units within the same layer.

Working Principle

17. Represent each user's movie history as a vector in the visible layer.
18. Feed the interaction vector into the RBM.
19. Hidden neurons learn latent patterns such as preference for action, comedy, science fiction or particular actors.
20. The RBM learns relationships between movies and hidden preference patterns during training.
21. During recommendation, provide the user's known movie interactions to the visible layer.
22. The RBM estimates the probability or preference for movies that the user has not yet watched.
23. Rank the predicted preferences and recommend the highest-scoring suitable movies.

Example

If a user frequently watches superhero and science-fiction movies, the hidden units may learn a corresponding preference pattern. The RBM can then assign high preference scores to other movies sharing similar learned characteristics.

Advantages

- Can learn hidden user-preference patterns automatically.
- Does not require manually labeled recommendation targets.
- Can support collaborative filtering.
- Can discover relationships between users and items.

Conclusion

An RBM can serve as a recommendation model by learning latent relationships between users and movies. It converts sparse viewing-history data into hidden preference representations and uses those representations to estimate suitable recommendations.

5. Deep Learning Applications – Representation Learning

Scenario

A manufacturing company collects sensor readings every second from machines operating on a production line. The sensors measure variables such as temperature, vibration, pressure, speed and current. The company wants to automatically detect defective products or abnormal machine behavior.

Proposed Deep Learning Solution

A neural-network-based system can be designed to learn useful representations from the time-series sensor data and classify or detect defective conditions. If labeled defect data is available, a supervised deep learning model can be used. If labels are limited, an Autoencoder-based anomaly detection approach can be used.

Workflow

24. Sensor Data Collection – Continuously collect temperature, vibration, pressure, speed and current readings.
25. Preprocessing – Remove noise, handle missing values, normalize the sensor measurements and divide the continuous stream into fixed time windows.
26. Representation Learning – Feed each time window into a neural network so that the model learns important patterns automatically rather than relying only on manually designed features.

27. Feature Extraction – A 1D CNN can learn local patterns in sensor signals, while an LSTM or similar sequence model can learn temporal relationships. A combined architecture can also be used when both local and time-dependent patterns matter.
28. Latent Representation – The network converts the raw sensor window into a compact representation containing important information about machine behavior.
29. Prediction/Detection – A classification layer can predict normal versus defective conditions when labeled data is available. Alternatively, an Autoencoder can reconstruct normal sensor patterns and use reconstruction error to identify anomalies.
30. Decision Stage – If the predicted defect probability or anomaly score exceeds a selected threshold, the product or machine state is flagged for inspection.
31. Continuous Improvement – New production data can be used to retrain or fine-tune the model as operating conditions change.

Example Architecture

Sensor Readings → Preprocessing → 1D CNN/LSTM → Learned Representation → Dense Layer → Defect Probability / Anomaly Score → Quality-Control Decision

Role of Representation Learning

- Automatically discovers useful patterns from raw sensor readings.
- Reduces dependence on manually engineered features.
- Can capture complex relationships among multiple sensors.
- Can identify subtle patterns associated with defective products.

Performance Evaluation

For a labeled classification system, suitable measures include accuracy, precision, recall, F1-score and confusion matrix. In a defect-detection problem, recall is particularly important because missing a defective product can have a significant operational or safety impact. For an Autoencoder-based system, reconstruction error and anomaly-detection metrics can be evaluated.

Conclusion

A deep learning solution can continuously process sensor data and learn representations of normal and defective operating patterns. A CNN/LSTM-based model is suitable when labeled data is available, while an Autoencoder is useful

when the company has mainly normal or unlabeled sensor data. This approach can improve automated quality inspection and support early detection of manufacturing problems.

Overall Assessment Summary

The five scenarios demonstrate how deep learning concepts can be applied to practical problems. Autoencoders are useful for unsupervised anomaly detection and dimensionality reduction, representation learning allows neural networks to discover useful features automatically, network depth and width control learning capacity and computational requirements, activation functions provide the required non-linearity, and RBMs can learn hidden relationships for recommendation systems. In manufacturing, these concepts can be combined to build intelligent systems that learn directly from sensor data.